

Sprawozdanie z Listy 3 (Laboratorium)

Obliczenia Naukowe

Agnieszka Głuszkiewicz

1 Zadanie 1: Metoda bisekcji

1.1 Opis problemu

Zadanie polegało na implementacji funkcji rozwiązującej równanie $f(x) = 0$ metodą bisekcji. Jest to iteracyjna metoda poszukiwania pierwiastka w zadanym przedziale $[a, b]$, która wymaga, aby funkcja ciągła miała różne znaki na końcach przedziału startowego.

1.2 Rozwiązanie

Zaimplementowałam funkcję `mbisekcji` zgodnie ze specyfikacją podaną w treści zadania. Testy do funkcji znajdują się w pliku `tests.jl`.

Dane:

- f – funkcja $f(x)$, dla której szukamy miejsca zerowego.
- a, b (`Float64`) – końce przedziału początkowego $[a, b]$, w którym spodziewamy się znaleźć pierwiastek.
- δ (`delta::Float64`) – dokładność obliczeń względem szerokości przedziału. Warunkiem stopu jest sytuacja, w której długość aktualnego przedziału jest mniejsza od δ .
- ϵ (`epsilon::Float64`) – dokładność obliczeń względem wartości funkcji. Algorytm kończy działanie, jeśli wartość funkcji w wyznaczonym punkcie spełnia warunek $|f(x)| < \epsilon$.

Wyniki:

Funkcja zwraca czwórkę (r, v, it, err) , gdzie:

- r – znalezione przybliżenie pierwiastka równania $f(x) = 0$.
- v – wartość funkcji w wyznaczonym punkcie przybliżenia, tj. $v = f(r)$.
- it – liczba iteracji wykonana przez algorytm do momentu spełnienia warunków stopu.
- err – kod błędu sygnalizujący status zakończenia obliczeń:
 - 0 – brak błędu.
 - 1 – funkcja nie zmienia znaku na końcach przedziału $[a, b]$.

1.3 Opis działania algorytmu

Przed rozpoczęciem iteracji funkcja weryfikuje znaki funkcji na krańcach przedziału $[a, b]$. Jeśli znaki są zgodne, funkcja natychmiast przerywa działanie zwracając kod błędu 1.

W przeciwnym wypadku zaczyna się w pętli proces iteracji:

1. Wyznaczany jest środek przedziału $r = a + \frac{b-a}{2}$.
2. Sprawdzane są warunki stopu: czy szerokość połowy przedziału z poprzedniego kroku jest mniejsza od δ lub czy wartość $|f(r)|$ jest mniejsza od ϵ .

3. Jeśli warunki są spełnione, zwracany jest wynik. Jeśli warunki nie są spełnione, przedział jest zawężany: wybierana jest ta połowa przedziału ($[a, r]$ lub $[r, b]$), na której końcach funkcja przyjmuje przeciwne znaki.

2 Zadanie 2: Metoda Newtona (stycznych)

2.1 Opis problemu

Zadanie polegało na implementacji funkcji rozwiązującej równanie $f(x) = 0$ metodą Newtona. Jest to metoda iteracyjna, wykorzystująca wartość funkcji oraz jej pochodnej w danym punkcie do wyznaczenia stycznej, której przecięcie z osią OX wyznacza nowe przybliżenie pierwiastka.

2.2 Rozwiązanie

Zaimplementowałam funkcję `mstycznych` (metodę Newtona), działającą według poniższej specyfikacji. Testy do funkcji znajdują się w pliku `tests.jl`.

Dane:

- f, pf – funkcja $f(x)$ oraz jej pochodną $f'(x)$.
- x_0 (`Float64`) – przybliżenie początkowe pierwiastka.
- δ (`delta::Float64`) – dokładność obliczeń względem argumentu (różnica między kolejnymi przybliżeniami). Warunek stopu: $|x_{k+1} - x_k| < \delta$.
- ϵ (`epsilon::Float64`) – dokładność obliczeń względem wartości funkcji. Warunek stopu: $|f(x_k)| < \epsilon$.
- $maxit$ (`Int`) – maksymalna dopuszczalna liczba iteracji.

Wyniki:

Funkcja zwraca czwórkę (r, v, it, err) , gdzie:

- r – znalezione przybliżenie pierwiastka.
- v – wartość funkcji w punkcie r , tj. $v = f(r)$.
- it – liczba wykonanych iteracji.
- err – kod błędu:
 - 0 – metoda zbieżna.
 - 1 – nie osiągnięto wymaganej dokładności w $maxit$ iteracji.
 - 2 – pochodna bliska zeru.

2.3 Opis działania algorytmu

Na początku funkcja sprawdza, czy punkt startowy x_0 jest już pierwiastkiem (spełnia warunek $|f(x_0)| < \epsilon$). Następnie w pętli (do $maxit$ razy) wykonywane są następujące kroki:

1. Obliczana jest wartość pochodnej w bieżącym punkcie. Jeśli jest bliska zeru, algorytm przerywa działanie zwracając błąd nr 2.
2. Wyznaczane jest nowe przybliżenie ze wzoru: $x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}$.
3. Sprawdzane są warunki stopu ($|x_1 - x_0| < \delta$ i $|f(x_1)| < \epsilon$). Jeśli któryś jest spełniony, funkcja zwraca wynik z kodem błędu 0.
4. W przeciwnym razie aktualizowane są zmienne ($x_0 \leftarrow x_1$) i następuje kolejna iteracja.

Jeśli pętla zakończy się bez spełnienia warunków stopu, zwracany jest kod błędu 1.

3 Zadanie 3: Metoda siecznych

3.1 Opis problemu

Celem zadania była implementacja funkcji rozwiązującej równanie $f(x) = 0$ metodą siecznych. Metoda ta przybliża funkcję sieczną przechodzącą przez dwa ostatnie punkty iteracji, nie wymagając przy tym znajomości wzoru na pochodną funkcji.

3.2 Rozwiązanie

Zaimplementowałam funkcję `msiecznych` zgodnie z poniższym opisem. Testy do funkcji znajdują się w pliku `tests.jl`.

Dane:

- f – funkcja $f(x)$.
- x_0, x_1 (`Float64`) – dwa przybliżenia początkowe niezbędne do rozpoczęcia algorytmu.
- δ (`delta::Float64`) – dokładność obliczeń względem różnicy kolejnych przybliżeń.
- ϵ (`epsilon::Float64`) – dokładność obliczeń względem wartości funkcji.
- $maxit$ (`Int`) – maksymalna liczba iteracji.

Wyniki:

Funkcja zwraca czwórkę (r, v, it, err) , gdzie:

- r – znalezione przybliżenie pierwiastka (x_k).
- v – wartość funkcji $f(r)$.
- it – liczba wykonanych iteracji.
- err – kod błędu:
 - 0 – metoda zbieżna.
 - 1 – nie osiągnięto wymaganej dokładności w $maxit$ iteracji.

3.3 Opis działania algorytmu

Algorytm działa iteracyjnie w pętli ograniczonej przez parametr $maxit$:

1. Na początku sprawdzany jest warunek $|f(x_1)| < |f(x_0)|$. Jeżeli jest spełniony, punkty x_0 i x_1 (oraz odpowiadające im wartości funkcji) są zamieniane miejscami.
2. Obliczana jest różnica wartości funkcji $f(x_1) - f(x_0)$ (mianownik we wzorze). Jeśli jest ona bliska zeru ($< eps(Float64)$), algorytm przerywa działanie zwracając błąd 1 (uniknięcie dzielenia przez zero).
3. Algorytm oblicza iloraz $s = \frac{x_1 - x_0}{f(x_1) - f(x_0)}$. Następnie wykonywane jest przesunięcie zmiennych:
 - Wartość poprzedniego przybliżenia jest zachowywana w zmiennych pomocniczych ($x_1 \leftarrow x_0$).
 - Obliczane jest nowe przybliżenie, które nadpisuje zmienną x_0 zgodnie ze wzorem:

$$x_0 = x_0 - f(x_0) \cdot s$$

- Obliczana jest wartość funkcji w nowym punkcie: $v_0 = f(x_0)$.
4. Po wykonaniu kroku sprawdzana jest zbieżność. Metoda kończy się sukcesem (kod 0), jeśli $|x_1 - x_0| < \delta$ lub $|f(x_0)| < \epsilon$.

Jeśli pętla zakończy się bez spełnienia warunków zbieżności, funkcja zwraca ostatnie obliczone przybliżenie z błędem 1.

4 Zadanie 4: Testowanie metod na równaniu $\sin(x) - (\frac{1}{2}x)^2 = 0$

4.1 Opis problemu

Celem zadania było przetestowanie zaimplementowanych wcześniej metod (bisekcji, Newtona i siecznych) na konkretnym przykładzie równania:

$$f(x) = \sin(x) - \left(\frac{1}{2}x\right)^2 = 0$$

Dla metody Newtona pochodna tej funkcji to

$$f'(x) = \cos(x) - \frac{x}{2}$$

Zadane dokładności obliczeń: $\delta = \frac{1}{2} \cdot 10^{-5}$ oraz $\epsilon = \frac{1}{2} \cdot 10^{-5}$.

4.2 Rozwiązanie

Dla każdej metody przyjęte parametry startowe są zgodne z treścią zadania:

- **Metoda bisekcji:** przedział początkowy $[1.5, 2.0]$.
- **Metoda Newtona:** przybliżenie początkowe $x_0 = 1.5$.
- **Metoda siecznych:** przybliżenia początkowe $x_0 = 1.0, x_1 = 2.0$.

4.3 Wyniki

Poniższa tabela przedstawia wyniki otrzymane dla poszczególnych metod. Wszystkie algorytmy zakończyły działanie sukcesem (kod błędu 0), osiągając zadaną dokładność.

Table 1: Porównanie wyników metod iteracyjnych dla $f(x) = \sin(x) - \frac{x^2}{4}$

Metoda	Przybliżenie r	Wartość f(r)	Liczba iteracji	Err
Bisekcji	1.9337539672851562	$-2.7027680138402843 \times 10^{-7}$	16	0
Newtona	1.933753779789742	$-2.2423316314856834 \times 10^{-8}$	4	0
Siecznych	1.933753644474301	$1.564525129449379 \times 10^{-7}$	4	0

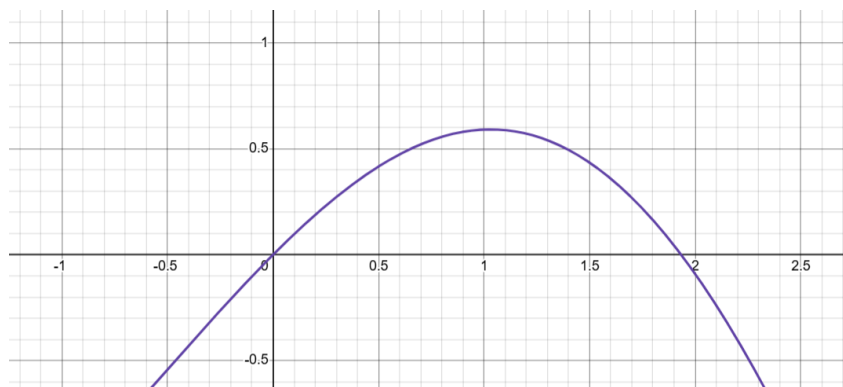


Figure 1: Fragment wykresu funkcji $f(x) = \sin(x) - (\frac{1}{2}x)^2$

4.4 Wnioski

- Po porównaniu z faktycznym poprawnym wynikiem można zauważyć, że wszystkie trzy metody poprawnie wyznaczyły przybliżenie pierwiastka $r \approx 1.93375$.
- Metody Newtona i siecznych okazały się znacznie szybsze od metody bisekcji, osiągając wymaganą dokładność już w 4 iteracjach. Metoda bisekcji, zbieżna liniowo, potrzebowała aż 16 iteracji, żeby znaleźć pierwiastek.
- Metoda Newtona pozwoliła uzyskać wynik, dla którego wartość funkcji była najbliższa zeru ($|f(r)| \approx 10^{-8}$), co potwierdza jej wysoką (kwadratową) zbieżność.

5 Zadanie 5: Punkty przecięcia wykresów $y = 3x$ i $y = e^x$

5.1 Opis problemu

Zadanie polegało na wyznaczeniu wartości zmiennej x , dla których przecinają się wykresy funkcji $y = 3x$ oraz $y = e^x$. Problem ten sprowadza się do znalezienia miejsc zerowych funkcji $f(x)$ będącej różnicą obu stron równania:

$$3x = e^x \implies f(x) = 3x - e^x = 0$$

Miejsca zerowe zostały wyznaczone metodą bisekcji z zadaną dokładnością obliczeń $\delta = 10^{-4}$ oraz $\epsilon = 10^{-4}$.

5.2 Rozwiązanie

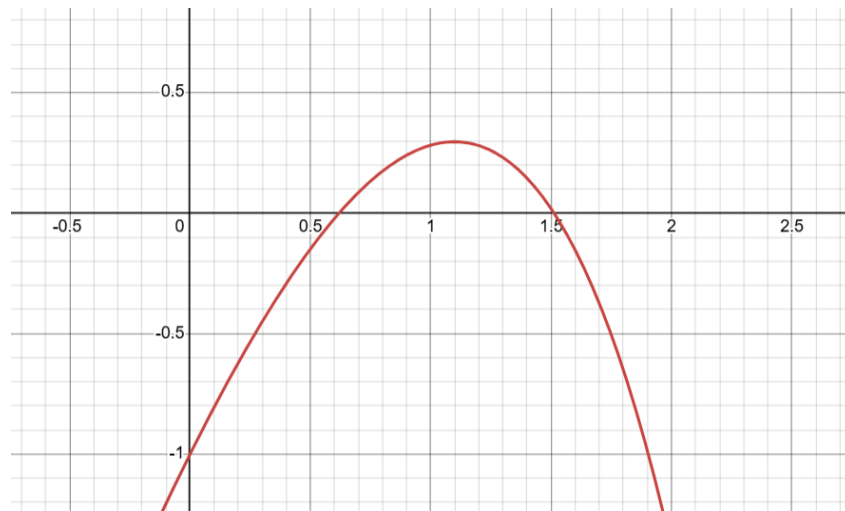


Figure 2: Fragment wykresu funkcji $f(x) = 3x - e^x$

Na podstawie analizy wykresu funkcji $f(x)$ określiłam dwa przedziały, w których funkcja zmienia znak:

1. Przedział $[0.5, 1.0]$
2. Przedział $[1.0, 2.0]$

Dla obu przedziałów zastosowałam zaimplementowaną wcześniej metodę bisekcji.

5.3 Wyniki

Poniższa tabela prezentuje wyniki zwrócone przez program dla wybranych wartości parametrów początkowych.

Table 2: Wyniki metody bisekcji dla funkcji $f(x) = 3x - e^x$

Przedział pocz.	Przybliżenie r	Wartość $f(r)$	Liczba iteracji	Err
$[0.5, 1.0]$	0.619140625	$9.066320343276146 \times 10^{-5}$	8	0
$[1.0, 2.0]$	1.5120849609375	$7.618578602741621 \times 10^{-5}$	13	0

5.4 Wnioski

- Funkcje $y = 3x$ i $y = e^x$ przecinają się w dwóch punktach. Metoda bisekcji skutecznie odnalazła oba rozwiązania w wybranych przedziałach.
- Wartości funkcji w znalezionych punktach (v) są rzędu 10^{-5} , co spełnia warunek stopu $|f(x)| < \epsilon = 10^{-4}$.
- Dla zadanego poziomu dokładności $\delta = 10^{-4}$, metoda potrzebowała niewielkiej liczby iteracji (odpowiednio 8 i 13), co wynika z faktu, że przedziały początkowe były stosunkowo wąskie, a żądana precyzja niezbyt rygorystyczna.

6 Zadanie 6: Miejsca zerowe funkcji f_1 i f_2

6.1 Opis problemu

Zadanie polegało na znalezieniu miejsc zerowych dwóch funkcji:

- $f_1(x) = e^{1-x} - 1$ (oczekiwane miejsce zerowe $x = 1$),
- $f_2(x) = xe^{-x}$ (oczekiwane miejsce zerowe $x = 0$),

przy użyciu metod bisekcji, Newtona i siecznych, z dokładnością $\delta = 10^{-5}$ i $\epsilon = 10^{-5}$. Masymalna liczba iteracji jest ustawiona na 100.

6.2 Analiza funkcji $f_1(x) = e^{1-x} - 1$

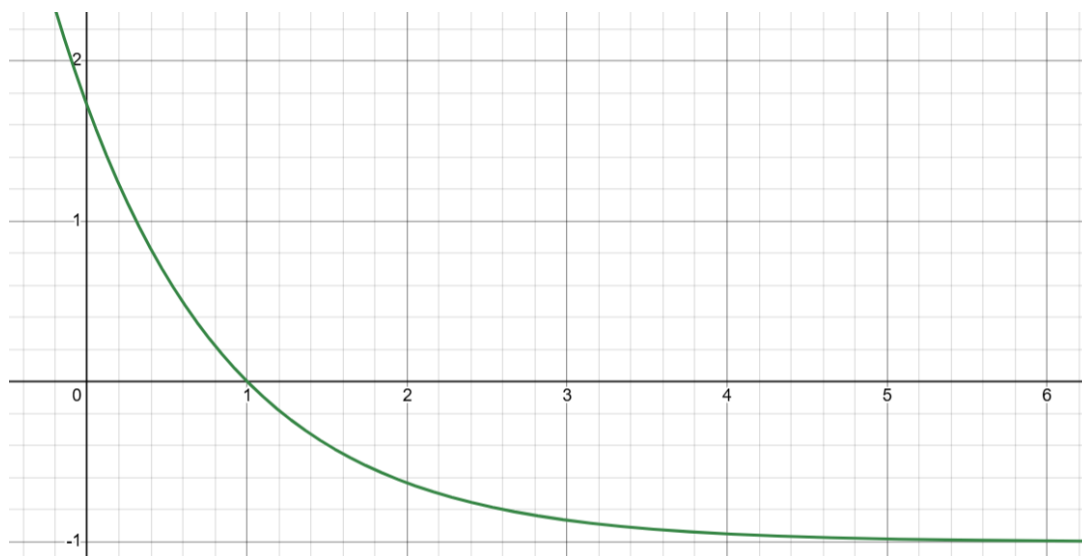


Figure 3: Fragment wykresu funkcji $f_1(x) = e^{1-x} - 1$

6.2.1 Metoda Bisekcji dla f_1

Table 3: Metoda Bisekcji dla $f_1(x)$

Przedział $[a, b]$	Wynik r	Wartość $f(r)$	Liczba iteracji	Err
$[0.0, 2.0]$	1.0	0.0	1	0
$[0.0, 3.0]$	1.0000076293945312	$-7.6293654275305656 \times 10^{-6}$	17	0
$[-100.0, 100.0]$	0.9999990463256836	$9.536747711536009 \times 10^{-7}$	23	0
$[-1.0, 1500.0]$	1.000002846121788	$-2.846117737820286 \times 10^{-6}$	26	0

Metoda bisekcji działa stabilnie dla każdego z rozpatrywanych przedziałów, w którym funkcja zmienia znak. W pierwszym przypadku ($[0, 2]$) środek przedziału wypadł idealnie w pierwiastku, stąd zakończenie w 1. iteracji z błędem zerowym. W pozostałych przypadkach metoda zbiegła do jedynki, osiągając wymaganą dokładność po nieco większej liczbie iteracji, nawet dla przypadku z bardzo odległym prawym końcem początkowego przedziału.

6.2.2 Metoda Newtona dla f_1

Table 4: Metoda Newtona dla $f_1(x)$

Start x_0	Wynik r	Wartość $f(r)$	Liczba iteracji	Err
0.5	0.999999998878352	$1.1216494399945987 \times 10^{-10}$	4	0
1.5	0.999999984736215	$1.5263785790864404 \times 10^{-9}$	4	0
10.0	NaN	NaN	100	1
-10.0	0.999999998781014	$1.2189871334555846 \times 10^{-10}$	15	0
-100.0	-0.2024165040950583	2.328149700587754	100	1
100.0	100.0	-1.0	1	2

Dla punktów bliskich rozwiązania (0.5, 1.5) oraz dla $x_0 = -10$, metoda jest zbieżna. Problemy pojawiają się dla $x_0 = 10.0$, gdzie funkcja posiada asymptotę poziomą $y = -1$, przez co pochodna jest bliska zera – algorytm zwrócił NaN. Dla jeszcze większego $x_0 = 100.0$ pochodna jest już na tyle blisko zera, że zwracany jest błąd nr 2. Dla bardzo dalekiego punktu startowego po lewej stronie $x_0 = -100.0$ metoda nie osiągnęła zbieżności w 100 iteracjach.

6.2.3 Metoda Siecznych dla f_1

Table 5: Metoda Siecznych dla $f_1(x)$

Start x_0, x_1	Wynik r	Wartość $f(r)$	Liczba iteracji	Err
0.5, 1.5	0.9999999624498374	$3.755016342310569 \times 10^{-8}$	5	0
-2.0, 2.0	1.0000000080618678	$-8.061867839970205 \times 10^{-9}$	8	0
0.0, 5.0	3.1820380425510413	-0.8871886182027999	3	0
-10.0, -7.0	0.9999997162784523	$2.8372158800138436 \times 10^{-7}$	17	0
8.0, 10.0	8.0	-0.9990881180344455	2	0
-100.0, -90.0	-21.15064638085144	$4.1677675955236554 \times 10^9$	100	1

Metoda działa poprawnie dla rozważanej funkcji, gdy punkty startowe otaczają pierwiastek lub są po jego lewej stronie (nie za daleko - dla wartości początkowych -100.0, -90.0 liczba iteracji nie jest wystarczająca do osiągnięcia dokładności). W przypadku punktów startowych położonych po prawej stronie (pary 0.0, 5.0 oraz 8.0, 10.0), algorytm zatrzymuje się z kodem błędu 0, ponieważ różnica między kolejnymi

przybliżeniami x jest mała (ze względu na płaski wykres funkcji), mimo że wartość funkcji jest daleka od zera (ok. -0.88 i -0.99).

6.3 Analiza funkcji $f_2(x) = xe^{-x}$

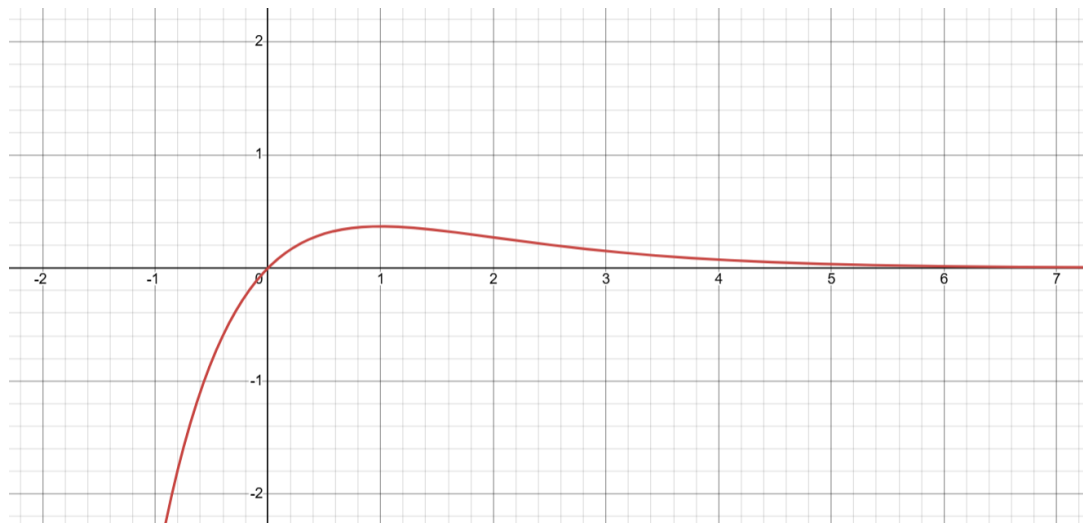


Figure 4: Fragment wykresu funkcji $f_2(x) = xe^{-x}$

6.3.1 Metoda Bisekcji dla f_2

Table 6: Metoda Bisekcji dla $f_2(x)$

Przedział $[a, b]$	Wynik r	Wartość $f(r)$	Liczba iteracji	Err
$[-1.0, 1.0]$	0.0	0.0	1	0
$[-1.0, 2.0]$	$7.62939453125 \times 10^{-6}$	$7.62933632381113 \times 10^{-6}$	17	0
$[-10.0, 11.0]$	$1.9073486328125 \times 10^{-6}$	$1.9073449948371624 \times 10^{-6}$	19	0
$[-10.0, 110.0]$	50.0	$9.643749239819589 \times 10^{-21}$	1	0

Dla przedziału symetrycznego $[-1, 1]$ metoda bisekcji trafia w zero w pierwszej iteracji. Dla dwóch kolejnych, nie za szerokich przedziałów również znajduje pierwiastki, po nieco większej liczbie iteracji. Z kolei dla przedziału, którego środek znajduje się mocno po prawej stronie, za pierwiastek jest uznawany argument, który nim nie jest - ale jego wartość bardzo bliska zera jest wystarczająca przy zastosowanej dokładności, by go za takowy uznać.

6.3.2 Metoda Newtona dla f_2

Table 7: Metoda Newtona dla $f_2(x)$

Start x_0	Wynik r	Wartość $f(r)$	Liczba iteracji	Err
-0.5	$-3.0642493416461764 \times 10^{-7}$	$-3.0642502806087233 \times 10^{-7}$	4	0
-10.0	$-3.784424932490619 \times 10^{-7}$	$-3.784426364678097 \times 10^{-7}$	16	0
0.5	$-3.0642493416461764 \times 10^{-7}$	$-3.0642502806087233 \times 10^{-7}$	5	0
1.0	1.0	0.36787944117144233	1	2
1.5	14.787436802837927	$5.594878975694858 \times 10^{-6}$	10	0
50.0	50.0	$9.643749239819589 \times 10^{-21}$	0	0
-100.0	-3.2905929088682004	-88.38132333845007	100	1

Dla nie za dużych $x_0 < 1$ metoda zbiega poprawnie do zera, ale np. już dla $x_0 = -100$ nie udaje się osiągnąć wymaganej dokładności. W punkcie $x_0 = 1$, gdzie funkcja ma ekstremum, pochodna wynosi zero, co skutkuje błędem nr 2. Dla $x_0 > 1$ (1.5) styczna ma ujemne nachylenie i wyrzuca kolejne przybliżenia w stronę dużych wartości dodatnich lub od razu uznaje argument za miejsce zerowe (50.0). Algorytm kończy wtedy działanie sukcesem, ponieważ wartość funkcji spada tam poniżej ϵ , mimo że nie jest to rzeczywiste miejsce zerowe.

6.3.3 Metoda Siecznych dla f_2

Table 8: Metoda Siecznych dla $f_2(x)$

Start x_0, x_1	Wynik r	Wartość $f(r)$	Liczba iteracji	Err
-0.5, 0.5	$5.38073548562323 \times 10^{-6}$	$5.380706533386756 \times 10^{-6}$	6	0
-0.5, 1.0	$7.367119728946578 \times 10^{-6}$	$7.3670654546934 \times 10^{-6}$	10	0
-5.0, -10.0	$-1.3163793904651081 \times 10^{-8}$	$-1.3163794077936554 \times 10^{-8}$	16	0
-10.0, 11.0	10.999999982484285	0.0001837187116181246	1	0
8.0, 9.0	14.289416415442075	$8.896102815708247 \times 10^{-6}$	7	0
-100.0, -90.0	-22.520006955048267	$-1.3579484174684215 \times 10^{11}$	100	1
30.0, 40.0	40.000605369041786	$1.6983389872333798 \times 10^{-16}$	1	0

Podobnie jak w metodzie Newtona, metoda siecznych zbiega do zera, gdy punkty startowe są blisko zera lub na lewo od ekstremum (ale nie bardzo daleko, dla $-100.0, -90.0$ metoda już sobie nie radzi). Dla pary $[-10.0, 11.0]$ algorytm w jednej iteracji przeskoczył do $x \approx 11$, uznając to za wynik (spełniając warunek $|x_1 - x_0| < \delta$ dla r i wartości 11). Dla punktów startowych daleko po prawej stronie, algorytm podąża wzdłuż asymptoty (do $x \approx 14$ dla 8.0, 9.0) lub od razu uznaje pierwszy wyliczony punkt za miejsce zerowe (dla 30.0, 40.0), gdzie wartość funkcji jest wystarczająco mała, by spełnić warunek stopu ϵ .

6.4 Wnioski

1. Metoda Newtona i metoda siecznych nie gwarantują zbieżności globalnej, ich skuteczność zależy od wyboru przybliżenia początkowego. Wybranie punktu startowego w obszarze, gdzie funkcja jest "płaska" (bliska asymptoty) lub posiada ekstremum, prowadzi do błędów lub zbieżności do niewłaściwych wartości.
2. Metoda Newtona nie może wystartować z punktu, w którym pochodna funkcji wynosi zero, co było widać dla f_2 przy ekstremum w $x_0 = 1$.
3. W przypadku funkcji posiadających asymptoty poziome (jak f_1 dla $x \rightarrow \infty$ czy f_2 dla $x \rightarrow \infty$), małe zmiany wartości funkcji w kolejnych krokach mogą sprawić, że punkt jest uznawany za miejsce zerowe, mimo że algorytm znajduje się daleko od rzeczywistego pierwiastka.
4. W przypadku argumentów, dla których przyjmowana jest bardzo mała wartość funkcji (bliska zeru), ich klasyfikacja jako miejsca zerowe może być błędna ze względu na przyjętą niewystarczającą dokładność.
5. Analiza przebiegu funkcji i dobranie właściwych parametrów początkowych jest bardzo ważne przy szukaniu miejsc zerowych funkcji.